

Register Number: 192425317

Name: M. Sahasra

CO1 – AT1

MLA0201 – Fundamentals of Machine Learning

Application - Based Questions

Q1) Implement the concept of probability distribution to predict the likelihood of spam emails in a mail filtering system.

A)

Introduction:

Spam emails are unwanted or unsolicited messages sent in bulk to users through email services. These emails often contain advertisements, phishing links, malware, or fraudulent content. Since millions of emails are exchanged every day, manually identifying spam emails is impractical. Therefore, Machine Learning (ML) techniques are widely used to automatically classify emails as Spam or Not Spam (Ham).

One of the fundamental concepts used in spam email prediction is Probability Distribution. A probability distribution helps determine the likelihood that an email belongs to a particular category based on its features such as keywords, sender information, links, attachments, and message frequency. By analysing historical email data, a probability-based machine learning model can accurately estimate whether a newly received email is likely to be spam.

The objective of this system is to use probability distributions to calculate the likelihood of an incoming email being spam and assist users in filtering unwanted messages efficiently.

Dataset Used:

Dataset Name: Spam Email Classification Dataset

Source: Kaggle – Spam Email Classification Dataset

Dataset Link: <https://www.kaggle.com/datasets/ashfakyeafi/spam-email-classification>

Dataset Description

The dataset contains thousands of labelled email messages categorized as either Spam or Ham (Not Spam).

Target Variable:

Label

- $1 \rightarrow \text{Spam}$
- $0 \rightarrow \text{Not Spam}$

Machine Learning Concepts:

Probability Distribution:

Definition:

A Probability Distribution is a mathematical function that describes the probability of occurrence of different possible outcomes of an event.

It helps answer questions like:

- What is the probability that an email is spam?
- What is the probability that an email is genuine?

In spam filtering, probability distributions estimate the likelihood that an email belongs to the spam class based on previously observed email data.

Types of Probability Distribution:

1. Discrete Probability Distribution

Used when outcomes are countable.

Example:

- Spam
- Not Spam

Only one outcome occurs.

2. Continuous Probability Distribution

Used for numerical values such as

- Email size
- Time between emails
- Number of words

Important Probability Terms:

Sample Space (S)

The set of all possible outcomes.

Example

$S = \{\text{Spam, Ham}\}$

Event (E)

An event is one possible outcome.

Example

$E = \text{Email is Spam}$

Probability Formula:

$$P(E) = \frac{\text{Number of favorable outcomes}}{\text{Total number of outcomes}}$$

Example:

Suppose

Total Emails = 1000

Spam Emails = 350

Then

$$P(\text{Spam}) = 350/1000 = 0.35$$

Meaning

There is a **35% probability** that an incoming email is spam.

Conditional Probability:

Sometimes probability depends upon another event.

Formula:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

Where,

$A = \text{Email is Spam}$

$B = \text{Email contains suspicious keywords}$

This probability becomes higher if spam-related keywords such as

- Win
- Lottery
- Prize

- Free Money

are present.

Working Principle:

The spam prediction system works in the following stages:

Step 1:

Collect historical email dataset.

Step 2:

Clean the data by removing

- punctuation
- special symbols
- stop words
- duplicate emails

Step 3:

Convert text into numerical features using techniques such as Bag of Words or TF-IDF.

Step 4:

Calculate probability distributions for Spam and Ham classes.

Step 5:

Compare the calculated probabilities.

Step 6:

Assign the email to the class having the highest probability.

Step 7:

Display the prediction result.

Methodology:

Step 1 – Data Collection

Download the Spam Email Dataset from Kaggle.

Step 2 – Data Preprocessing

- Remove missing values
- Remove duplicate emails
- Convert text to lowercase

- Remove stop words
- Perform tokenization

Step 3 – Feature Extraction

Transform email text into numerical vectors using TF-IDF or Count Vectorization.

Step 4 – Probability Calculation

Calculate

- Probability of Spam
- Probability of Ham

using the frequency of features.

Step 5 – Prediction

Predict whether a new email is

- Spam
- Not Spam

based on the higher calculated probability.

Step 6 – Model Evaluation

Evaluate the prediction using metrics such as

- Accuracy
- Precision
- Recall
- F1-Score

Algorithm (Pseudocode):

Spam Email Prediction Using Probability Distribution:

1. Start the program.
2. Load the Spam Email Classification dataset.
3. Read email messages and corresponding labels.
4. Preprocess the text by removing punctuation, stop words, and converting to lowercase.
5. Convert the processed text into numerical features.
6. Split the dataset into training and testing sets.
7. Calculate the probability distribution for Spam and Ham classes.
8. Compute the probability of the test email belonging to each class.
9. Compare both probabilities.

10. If Spam probability is greater than Ham probability, classify the email as Spam.
11. Otherwise, classify it as Not Spam.
12. Display the prediction and evaluation metrics.
13. Stop.

Python Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

data = pd.read_csv("spam.csv")
X = data["Message"]
y = data["Label"]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = MultinomialNB()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, prediction))
```

Sample Input:

Email Subject	- Email Message
Congratulations!	- You have won ₹10,00,000. Click here to claim your prize.

Sample Output:

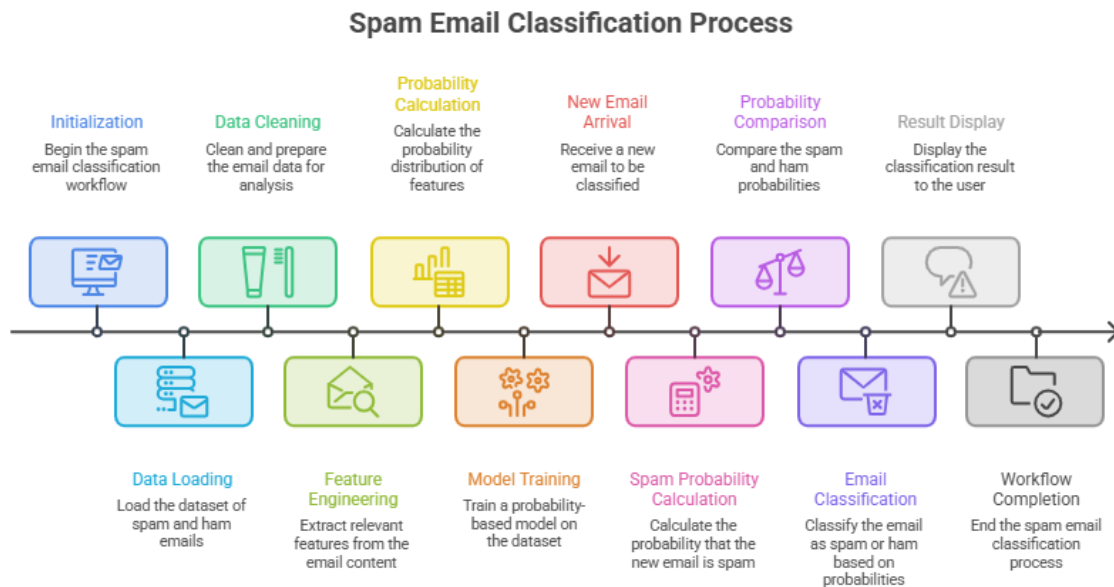
Predicted Class: Spam

Spam Probability: 0.96

Ham Probability: 0.04

Accuracy: 98%

Flowchart:



Advantages:

- Automatically filters unwanted emails.
- Reduces phishing and cyberattack risks.
- Saves users' time by eliminating manual filtering.
- Works efficiently on large datasets.
- Learns from historical email patterns.
- Improves email security.
- Can be integrated into modern email services.
- Provides high prediction accuracy with appropriate training data.

Limitations:

- Requires a large labeled dataset for effective learning.
- New spam techniques may reduce accuracy until the model is retrained.
- Incorrect preprocessing can affect prediction quality.
- Some genuine emails may be wrongly classified as spam (false positives).
- Performance depends on feature quality and dataset balance.

Real-World Applications:

- Gmail Spam Filter
- Microsoft Outlook Email Protection
- Yahoo Mail Spam Detection
- Enterprise Email Security Systems
- Banking Email Fraud Detection
- Educational Institution Email Filtering
- Corporate Communication Platforms
- Cloud-Based Email Security Solutions

Conclusion

Probability distribution is a fundamental concept in machine learning that enables the prediction of spam emails by estimating the likelihood of different outcomes. By analyzing historical email data and calculating the probability of an email belonging to the Spam or Ham class, the system can automatically filter unwanted messages with high accuracy. The proposed spam email prediction system involves data collection, preprocessing, feature extraction, probability calculation, model training, prediction, and evaluation. Such systems improve email security, reduce manual effort, and protect users from phishing attacks and malicious content, making probability-based spam filtering an essential application of machine learning in modern communication systems.

Q3) Develop the Design Steps of a Learning System to Build a Machine Learning Solution for Student Performance Prediction

A)

Introduction:

Student Performance Prediction is one of the most important applications of Machine Learning in the education sector. Educational institutions collect large amounts of student data such as attendance, assignment scores, internal assessment marks, study hours, participation in extracurricular activities, and previous academic performance. By analysing these data, machine learning models can predict whether a student is likely to perform well or require additional academic support.

A Learning System in Machine Learning follows a systematic process of collecting data, preprocessing it, training a model, testing the model, and making predictions. Designing a proper learning system ensures that the model learns patterns effectively and produces accurate predictions.

The objective of this system is to design a complete machine learning workflow that predicts student performance based on academic and behavioural factors, enabling teachers and institutions to provide timely guidance and improve learning outcomes.

Dataset Used:

Dataset Name: Student Performance Prediction Dataset

Source: Kaggle – Student Performance Dataset / UCI Machine Learning Repository

Dataset Link: <https://www.kaggle.com/datasets/ashfakyeafi/spam-email-classification>

Dataset Description

The dataset contains academic and personal information of students along with their final performance.

Target Variable:**Final_Performance**

Possible classes:

- Excellent
- Good
- Average
- Poor

Machine Learning Concepts:**Learning System:****Definition:**

A Learning System is a structured process through which a machine learns patterns from historical data and uses those learned patterns to make predictions or decisions on new, unseen data. Instead of being explicitly programmed for every situation, the system improves its performance automatically by analysing examples.

A learning system generally consists of the following components:

- Data Collection
- Data Preprocessing
- Feature Selection
- Model Training
- Model Testing
- Performance Evaluation
- Prediction

Supervised Learning:

Student performance prediction is a Supervised Learning problem because the dataset contains both input features and known output labels (student performance categories).

The model learns from labelled examples and predicts the performance of new students.

Features and Target Variable:

Features (Independent Variables):

- Study Hours
- Attendance
- Assignment Marks
- Internal Marks
- Previous Grade
- Participation

Target Variable (Dependent Variable):

- Final Performance

The model learns the relationship between the input features and the target variable.

Mathematical Representation:

A supervised learning model can be represented as:

$$Y = f(X)$$

Where:

- **X** = Input Features
- **Y** = Predicted Output
- **f** = Machine Learning Model

For student performance prediction:

$$Performance = f(\text{Study hours, Attendance, Marks, Participation})$$

The objective is to minimize prediction errors and maximize model accuracy.

Data Splitting Formula:

The dataset is divided into:

- Training Set = 80%

- Testing Set = 20%

Training data is used to teach the model, while testing data evaluates its performance on unseen data.

Design Steps of the Learning System:

Step 1 – Problem Definition

The first step is to clearly define the problem.

Problem Statement:

"Predict the academic performance of students using their educational records."

The expected output is a performance category such as Excellent, Good, Average, or Poor.

Step 2 – Data Collection

Collect historical student records from schools or colleges.

The dataset may include:

- Attendance records
- Study hours
- Assignment marks
- Internal examination marks
- Previous semester grades
- Classroom participation

A larger and more diverse dataset generally improves prediction accuracy.

Step 3 – Data Preprocessing

Raw educational data often contains missing values, duplicate records, or inconsistent formats. Before training the model, the data must be cleaned.

Preprocessing tasks include:

- Removing duplicate records
- Filling missing values
- Converting categorical values into numerical form
- Normalizing numerical values
- Removing irrelevant attributes

Proper preprocessing enhances the quality of the data and improves model performance.

Step 4 – Feature Selection

Not every feature contributes equally to prediction. Relevant features are selected based on their importance.

Examples of useful features include:

- Study Hours
- Attendance
- Assignment Marks
- Internal Marks
- Previous Grade

Removing unnecessary features reduces computation time and improves model efficiency.

Step 5 – Splitting the Dataset

The processed dataset is divided into two parts:

- **Training Dataset (80%)** – Used to train the model.
- **Testing Dataset (20%)** – Used to evaluate model performance.

This prevents overfitting and ensures reliable evaluation.

Step 6 – Model Training

The selected machine learning algorithm is trained using the training dataset.

Common algorithms include:

- Decision Tree
- Random Forest
- Logistic Regression
- Support Vector Machine (SVM)
- Naïve Bayes

The algorithm learns relationships between student characteristics and academic performance.

Step 7 – Prediction

After training, the model predicts the performance of new students.

Example:

Student Data:

- Attendance = 92%

- Study Hours = 4 hours/day
- Assignment Marks = 88
- Internal Marks = 90

Predicted Output:

Excellent

Step 8 – Performance Evaluation

The model is evaluated using performance metrics such as:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

Higher values indicate better prediction performance.

Step 9 – Deployment

After satisfactory evaluation, the trained model can be integrated into:

- School Management Systems
- Learning Management Systems (LMS)
- Student Monitoring Applications
- Academic Analytics Dashboards

This allows institutions to continuously monitor and improve student outcomes.

Methodology:

1. Collect the Student Performance dataset.
2. Clean and preprocess the data.
3. Select relevant features.
4. Split the dataset into training and testing sets.
5. Train the machine learning model.
6. Test the model using unseen data.
7. Predict student performance.
8. Evaluate the model using accuracy and other metrics.

9. Deploy the model for real-world educational use.

Algorithm (Pseudocode):

Student Performance Prediction Using a Learning System

1. Start the program.
2. Load the Student Performance dataset.
3. Read all student records and performance labels.
4. Handle missing values and remove duplicate records.
5. Convert categorical data into numerical values.
6. Select important academic features.
7. Split the dataset into training and testing sets.
8. Train the machine learning model using the training data.
9. Use the trained model to predict student performance for the testing data.
10. Compare predicted results with actual results.
11. Calculate evaluation metrics such as accuracy, precision, recall, and F1-score.
12. Display the prediction results and performance metrics.
13. Stop the program.

Python Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = pd.read_csv("student_performance.csv")

# Input features
X = data[["Study_Hours", "Attendance", "Assignment_Marks",
         "Internal_Marks", "Previous_Grade"]]

# Target variable
y = data["Final_Performance"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y, test_size=0.2, random_state=42)

# Train model
model = RandomForestClassifier()
model.fit(X_train, y_train)

# Prediction
prediction = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, prediction)
print("Accuracy:", accuracy)

```

Sample Input:

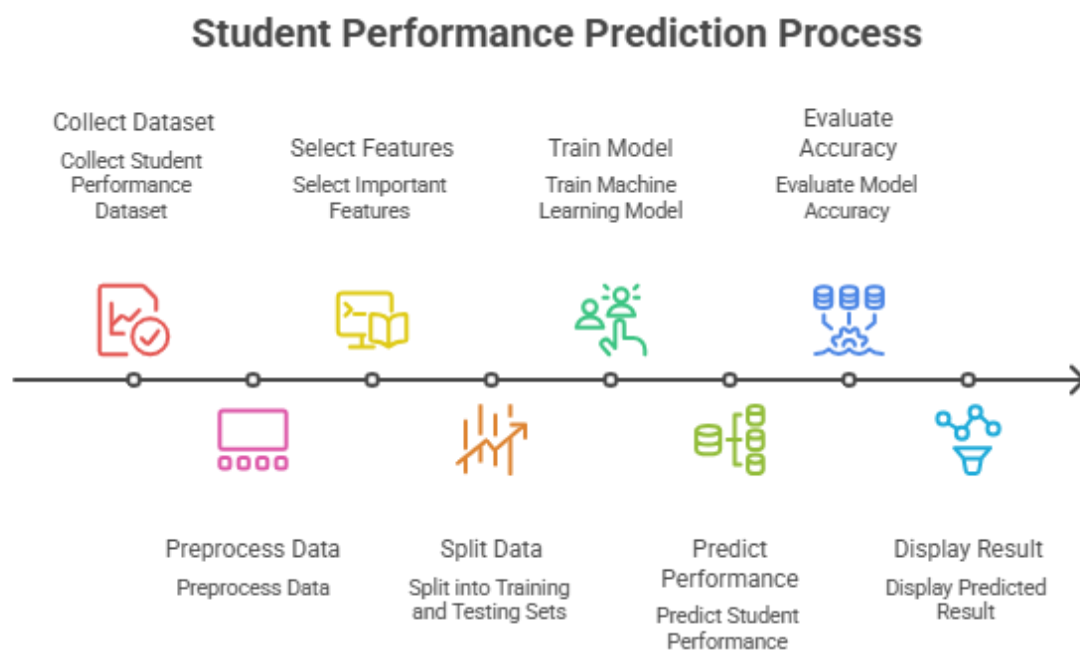
Study Hours	Attendance	Assignment Marks	Internal Marks	Previous Grade
5 Hours	94%	88	90	A

Sample Output:

Predicted Performance: Excellent

Model Accuracy: 94.8%

Flowchart:



Explanation of the Flowchart:

- Start: Begin the prediction process.
- Collect Dataset: Load historical student records.
- Preprocess Data: Clean and prepare the dataset.
- Feature Selection: Choose important academic attributes.
- Split Dataset: Divide data into training and testing sets.
- Train Model: Teach the algorithm using training data.
- Prediction: Predict the performance of new students.
- Evaluation: Measure model performance using accuracy and other metrics.
- Display Result: Show the predicted performance.
- End: Complete the process.

Advantages:

- Helps identify academically weak students at an early stage.
- Enables personalized learning strategies.
- Assists teachers in monitoring student progress.
- Improves educational planning and decision-making.
- Supports data-driven interventions.
- Increases overall academic performance.
- Reduces student dropout rates.
- Can be integrated with modern educational management systems.

Limitations:

- Prediction quality depends on the accuracy of the collected data.
- Missing or incorrect records may reduce model performance.
- Student behaviour may change over time, requiring periodic model retraining.
- External factors such as health or family issues may not be captured in the dataset.
- Biased datasets can lead to unfair predictions.

Real-World Applications:

- Student Performance Monitoring Systems
- School and College Learning Management Systems (LMS)

- Academic Counselling Platforms
- Scholarship Eligibility Prediction
- Early Warning Systems for At-Risk Students
- Personalized Learning Recommendation Systems
- Educational Analytics Dashboards
- Smart Education and E-Learning Platforms

Conclusion:

A well-designed machine learning system provides an effective approach for predicting student performance using academic and behavioural data. By following systematic steps such as data collection, preprocessing, feature selection, model training, prediction, and evaluation, the system can accurately identify students who may require additional academic support. Such predictive models help educators make informed decisions, improve teaching strategies, and enhance student success. Integrating these learning systems into educational institutions promotes data-driven decision-making and contributes to improved learning outcomes and overall academic excellence.

Q4) Model the Outcome of a Weather Forecasting Problem Using Random Variables and Probability Theory

A)

Introduction:

Weather forecasting is one of the most common real-world applications of Machine Learning, Probability Theory, and Statistics. Every day, weather departments collect vast amounts of data from satellites, weather stations, radar systems, and sensors to predict atmospheric conditions such as rainfall, temperature, humidity, wind speed, and cloud cover. Since weather conditions are uncertain and influenced by many environmental factors, probability theory is used to estimate the likelihood of different weather events.

A Random Variable represents uncertain whether outcomes, while Probability Theory quantifies the likelihood of these outcomes. By analysing historical weather data and current atmospheric conditions, machine learning models can predict whether it will rain, remain sunny, or become cloudy.

The objective of this system is to model a weather forecasting problem using random variables and probability concepts to estimate future weather conditions accurately.

Dataset Used:

Dataset Name: Weather Prediction Dataset

Source: Kaggle – Weather Prediction Dataset / Australian Weather Dataset / NOAA Weather Dataset

Dataset Link: <https://www.kaggle.com/datasets/muthuj7/weather-dataset>

Dataset Description

The dataset contains historical weather observations collected over several years.

Target Variable:

Weather Condition:

Possible classes:

- Rainy
- Sunny
- Cloudy

Machine Learning Concepts:

Random Variable:

Definition:

A Random Variable is a numerical value assigned to the outcome of a random experiment. It represents uncertain events whose values depend on chance.

In weather forecasting, examples of random variables include:

- Rainfall amount
- Daily temperature
- Wind speed
- Humidity level
- Atmospheric pressure

Since these values change unpredictably, they are modelled as random variables.

Types of Random Variables:

1. Discrete Random Variable

A discrete random variable has countable outcomes.

Example:

Weather Condition

- Sunny
- Rainy

- Cloudy

Only one outcome occurs at a particular time.

2. Continuous Random Variable

A continuous random variable can take infinitely many numerical values.

Examples include:

- Temperature
- Wind Speed
- Rainfall
- Humidity
- Atmospheric Pressure

Probability Theory:

Definition

Probability Theory is the branch of mathematics that measures the likelihood of an event occurring. It assigns a value between **0 and 1**, where:

- **0** indicates an impossible event.
- **1** indicates a certain event.

For weather forecasting, probability helps estimate the chance of events such as rainfall or sunshine.

Probability Formula:

The probability of an event **E** is calculated as:

$$P(E) = \frac{\text{Number of Favorable Outcomes}}{\text{Total Number of Outcomes}}$$

Where:

- $P(E)$ = Probability of event E
- Favorable Outcomes = Number of occurrences of the event
- Total Outcomes = Total observations

Example:

Suppose historical data contains:

- Total Days = 365

- Rainy Days = 120

Then,

$$P(Rain) = \frac{120}{365} = 0.329$$

Therefore,

Probability of Rain = 32.9%

Conditional Probability:

Weather prediction often depends on existing atmospheric conditions.

Conditional Probability is expressed as:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

Where:

- A = Rain occurs
- B = High Humidity

If humidity is above 90%, the probability of rainfall generally increases.

Working Principle:

The weather prediction system follows these stages:

Step 1: Collect historical weather observations.

Step 2: Preprocess the collected data by removing missing values and correcting inconsistencies.

Step 3: Identify important weather features such as temperature, humidity, pressure, and wind speed.

Step 4: Represent these features as random variables.

Step 5: Calculate probability distributions using historical observations.

Step 6: Train the machine learning model.

Step 7: Predict the weather condition for new observations.

Step 8: Evaluate prediction accuracy.

Methodology:

Step 1 – Data Collection

Historical weather information is collected from meteorological departments or public datasets.

The collected data includes:

- Temperature
- Humidity
- Wind Speed
- Rainfall
- Pressure
- Cloud Cover

Step 2 – Data Preprocessing

Before training the model:

- Remove duplicate records.
- Fill missing values.
- Normalize numerical features.
- Convert categorical values into numerical labels if necessary.

This improves the quality of the dataset.

Step 3 – Feature Selection

Select the most influential weather parameters.

Examples:

- Temperature
- Humidity
- Pressure
- Wind Speed
- Rainfall

These features strongly influence weather conditions.

Step 4 – Probability Modelling

Treat each weather parameter as a random variable.

Use historical data to estimate probabilities such as:

- Probability of Rain
- Probability of Sunny Weather

- Probability of Cloudy Weather

Step 5 – Model Training

Train a machine learning classifier such as:

- Decision Tree
- Random Forest
- Naïve Bayes
- Logistic Regression

The model learns relationships between weather variables and future weather conditions.

Step 6 – Prediction

Input today's weather observations into the trained model.

Example:

Temperature = 27°C

Humidity = 91%

Pressure = 1008 hPa

Wind Speed = 18 km/h

Predicted Weather:

Rainy

Step 7 – Model Evaluation

Evaluate performance using:

- Accuracy
- Precision
- Recall
- F1-Score

A higher accuracy indicates better forecasting performance.

Algorithm (Pseudocode):

Weather Forecast Prediction Using Random Variables and Probability

1. Start the program.
2. Load the Weather Prediction dataset.

3. Read all weather observations.
4. Remove missing and duplicate values.
5. Select important weather features.
6. Represent the features as random variables.
7. Calculate probability distributions from historical weather data.
8. Split the dataset into training and testing sets.
9. Train the machine learning model.
10. Predict future weather conditions using testing data.
11. Calculate model accuracy and evaluation metrics.
12. Display predicted weather conditions.
13. Stop the program.

Python Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = pd.read_csv("weather.csv")

# Input features
X = data[["Temperature", "Humidity", "Pressure", "WindSpeed"]]

# Target variable
y = data["Weather"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Train model
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# Prediction
prediction = model.predict(X_test)

# Accuracy
```

```
accuracy = accuracy_score(y_test, prediction)
print("Accuracy:", accuracy)
```

Sample Input:

Temperature	Humidity	Pressure	Wind Speed
27°C	91%	1008 hPa	18 km/h

Sample Output:

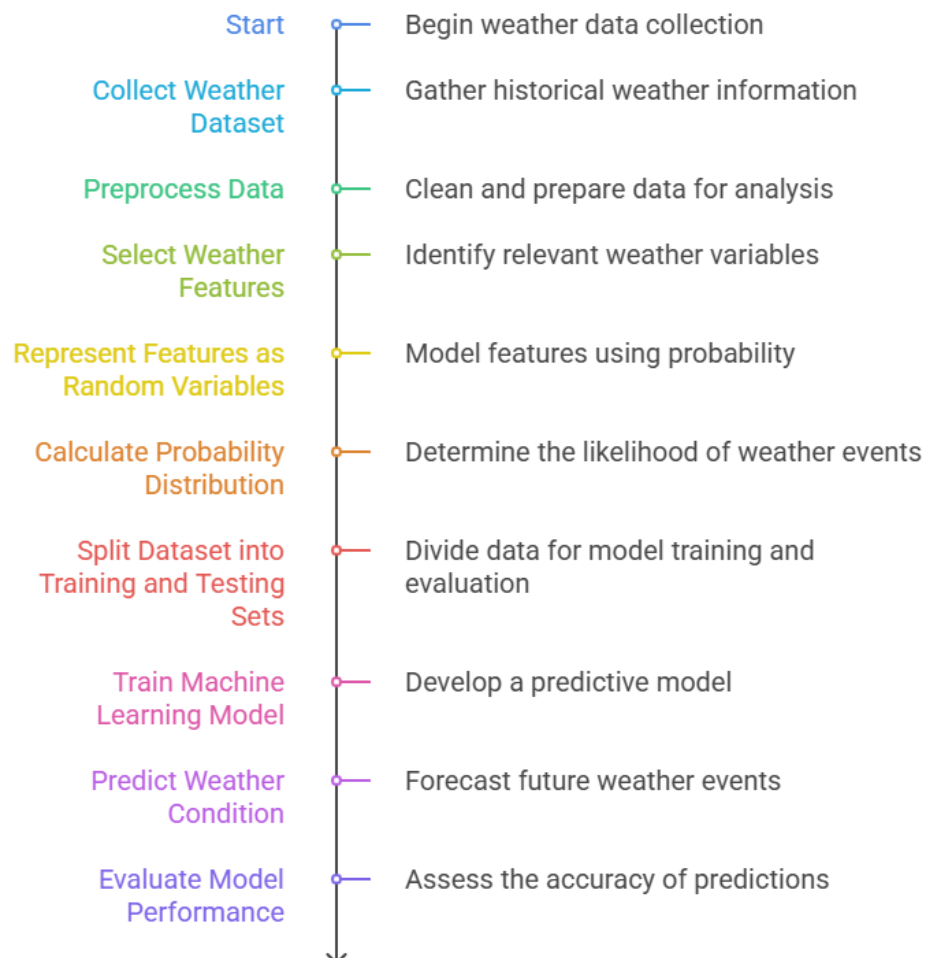
Predicted Weather: Rainy

Probability of Rain: 0.87

Model Accuracy: 95.2%

Flowchart:

Weather Prediction Model Development Timeline



Explanation of the Flowchart:

- Start: Begin the weather prediction process.
- Collect Dataset: Load historical weather observations.
- Preprocess Data: Clean and prepare the dataset.
- Select Features: Choose significant weather parameters.
- Random Variables: Represent uncertain weather values mathematically.
- Probability Distribution: Estimate the likelihood of different weather conditions.
- Train Model: Build the machine learning model.
- Prediction: Forecast future weather conditions.
- Evaluation: Measure the model's accuracy.
- Display Result: Show the predicted weather and probability.
- End: Complete the forecasting process.

Advantages:

- Provides early warning for adverse weather conditions.
- Improves disaster preparedness and risk management.
- Helps farmers plan agricultural activities.
- Assists airlines and transportation systems in route planning.
- Supports water resource management.
- Enhances decision-making using historical weather patterns.
- Can continuously improve with additional weather data.
- Enables accurate and automated forecasting.

Limitations:

- Prediction accuracy depends on the quality and quantity of historical weather data.
- Sudden atmospheric changes may reduce forecasting accuracy.
- Missing or incorrect sensor data can affect model performance.
- Weather systems are highly dynamic and may require frequent model updates.
- Computational resources increase with larger datasets and complex models.

Real-World Applications:

- National Meteorological Departments

- Agriculture and Crop Management
- Aviation Weather Forecasting
- Marine and Coastal Weather Services
- Disaster Management Systems
- Smart City Environmental Monitoring
- Renewable Energy Forecasting (Solar and Wind)
- Mobile Weather Forecasting Applications

Conclusion:

Weather forecasting using random variables and probability theory is an effective approach for modelling uncertain atmospheric conditions. By representing weather parameters as random variables and estimating the likelihood of different outcomes through probability distributions, machine learning models can accurately predict future weather conditions. The proposed system follows a structured learning process involving data collection, preprocessing, feature selection, probability modelling, model training, prediction, and evaluation. Such probability-based forecasting systems play a significant role in disaster management, agriculture, transportation, and environmental monitoring by providing reliable and timely weather predictions.

Q6) Apply Conditional Probability Concepts to Design a Fraud Detection Model for Suspicious Banking Transactions

A)

Introduction:

With the rapid growth of digital banking, online payments, and mobile transactions, financial fraud has become a major challenge for banks and financial institutions. Fraudulent transactions can lead to significant financial losses and compromise customer trust. Since millions of banking transactions occur every day, manually identifying fraudulent activities is nearly impossible. Therefore, Machine Learning and Probability Theory are widely used to automatically detect suspicious transactions.

One of the most important concepts used in fraud detection is Conditional Probability. It helps determine the probability that a transaction is fraudulent based on specific conditions such as unusually high transaction amounts, unusual locations, multiple failed login attempts, or abnormal spending patterns.

The objective of this system is to design a fraud detection model using conditional probability that predicts whether a banking transaction is genuine or fraudulent, allowing banks to take preventive action before financial loss occurs.

Dataset Used:

Dataset Name: Credit Card Fraud Detection Dataset

Source: Kaggle – Credit Card Fraud Detection Dataset

Dataset Link: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

Dataset Description:

The dataset contains thousands of banking and credit card transactions collected from customers.

Target Variable:**Transaction_Status**

Possible Classes:

- Fraud
- Genuine

Machine Learning Concepts:**Conditional Probability:****Definition:**

Conditional Probability is the probability of an event occurring given that another event has already occurred.

Unlike simple probability, conditional probability considers additional information before making a prediction.

In banking systems, the probability of fraud increases when suspicious conditions are observed.

Formula of Conditional Probability:

The mathematical formula is

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

Where:

- $P(A|B)$ = Probability of event A given event B
- $P(A \cap B)$ = Probability that both A and B occur together
- $P(B)$ = Probability of event B

Application in Fraud Detection:

Let

- A = Transaction is Fraudulent
- B = Transaction Amount is Greater than ₹50,000

Then

$$P(\text{Fraud} | \text{High Amount}) = \frac{P(\text{Fraud} \cap \text{High Amount})}{P(\text{High Amount})}$$

This calculates the probability that a transaction is fraudulent when the transferred amount is unusually high.

Example Calculation:

Suppose a bank records:

- Total Transactions = 10,000
- High Amount Transactions = 800
- Fraudulent High Amount Transactions = 200

Then,

$$P(\text{Fraud} | \text{High Amount}) = \frac{200}{800} = 0.25$$

Therefore,

Probability of Fraud = 25%

This indicates that one out of every four high-value transactions may be fraudulent.

Random Variables Used:

Some important random variables in banking fraud detection are:

- Transaction Amount
- Transaction Time
- Customer Location
- Device Used
- Login Attempts
- Merchant Category

These variables help the model identify unusual transaction behavior.

Working Principle:

The fraud detection system works in the following stages:

- Step 1:** Collect historical banking transaction data.
- Step 2:** Preprocess the dataset by removing missing values and duplicate records.
- Step 3:** Select important transaction features.
- Step 4:** Calculate conditional probabilities for suspicious events.
- Step 5:** Train the fraud detection model.
- Step 6:** Receive a new banking transaction.
- Step 7:** Calculate the probability that the transaction is fraudulent.
- Step 8:** Classify the transaction as Genuine or Fraud.
- Step 9:** Alert the bank if fraud is detected.

Methodology:

Step 1 – Data Collection

Collect transaction data from banking systems.

Important attributes include:

- Transaction Amount
- Time
- Location
- Device Used
- Previous Fraud History
- Merchant Category

Step 2 – Data Preprocessing

The collected data is cleaned by:

- Removing duplicate transactions.
- Handling missing values.
- Encoding categorical variables.
- Normalizing numerical values.

Proper preprocessing improves prediction accuracy.

Step 3 – Feature Selection

Select significant fraud indicators such as:

- Large transaction amount

- Unusual location
- Midnight transactions
- Multiple failed login attempts
- New device usage

These features strongly influence fraud prediction.

Step 4 – Probability Calculation

Using historical data, calculate:

- Probability of Fraud
- Probability of Genuine Transaction
- Conditional Probability based on suspicious conditions

These probabilities help estimate fraud risk.

Step 5 – Model Training

Train the machine learning model using algorithms such as:

- Logistic Regression
- Decision Tree
- Random Forest
- Naïve Bayes

The model learns normal and abnormal transaction patterns.

Step 6 – Prediction

Example Input:

Transaction Amount = ₹75,000

Location = Different Country

Time = 2:15 AM

Previous Fraud = Yes

Predicted Output:

Fraudulent Transaction

Step 7 – Evaluation

Evaluate model performance using:

- Accuracy
- Precision
- Recall
- F1 Score

Banks generally focus on achieving high recall so that fraudulent transactions are not missed.

Algorithm (Pseudocode):

Fraud Detection Using Conditional Probability

1. Start the program.
2. Load the Credit Card Fraud Detection dataset.
3. Read all transaction records.
4. Remove duplicate and incomplete records.
5. Select important transaction features.
6. Split the dataset into training and testing sets.
7. Calculate conditional probabilities for fraud-related events.
8. Train the machine learning model.
9. Receive a new banking transaction.
10. Calculate the probability that the transaction is fraudulent.
11. If fraud probability exceeds the predefined threshold, classify the transaction as Fraud.
12. Otherwise, classify it as Genuine.
13. Display prediction results and evaluation metrics.
14. Stop the program.

Python Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load dataset
data = pd.read_csv("creditcard_fraud.csv")
```

```

# Input features
X = data[["Transaction_Amount",
          "Transaction_Time",
          "Previous_Fraud"]]

# Target variable
y = data["Transaction_Status"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Train model
model = GaussianNB()
model.fit(X_train, y_train)

# Prediction
prediction = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, prediction)
print("Accuracy:", accuracy)

```

Sample Input:

Transaction Amount	Transaction Time	Previous Fraud
₹75,000	02:15 AM	Yes

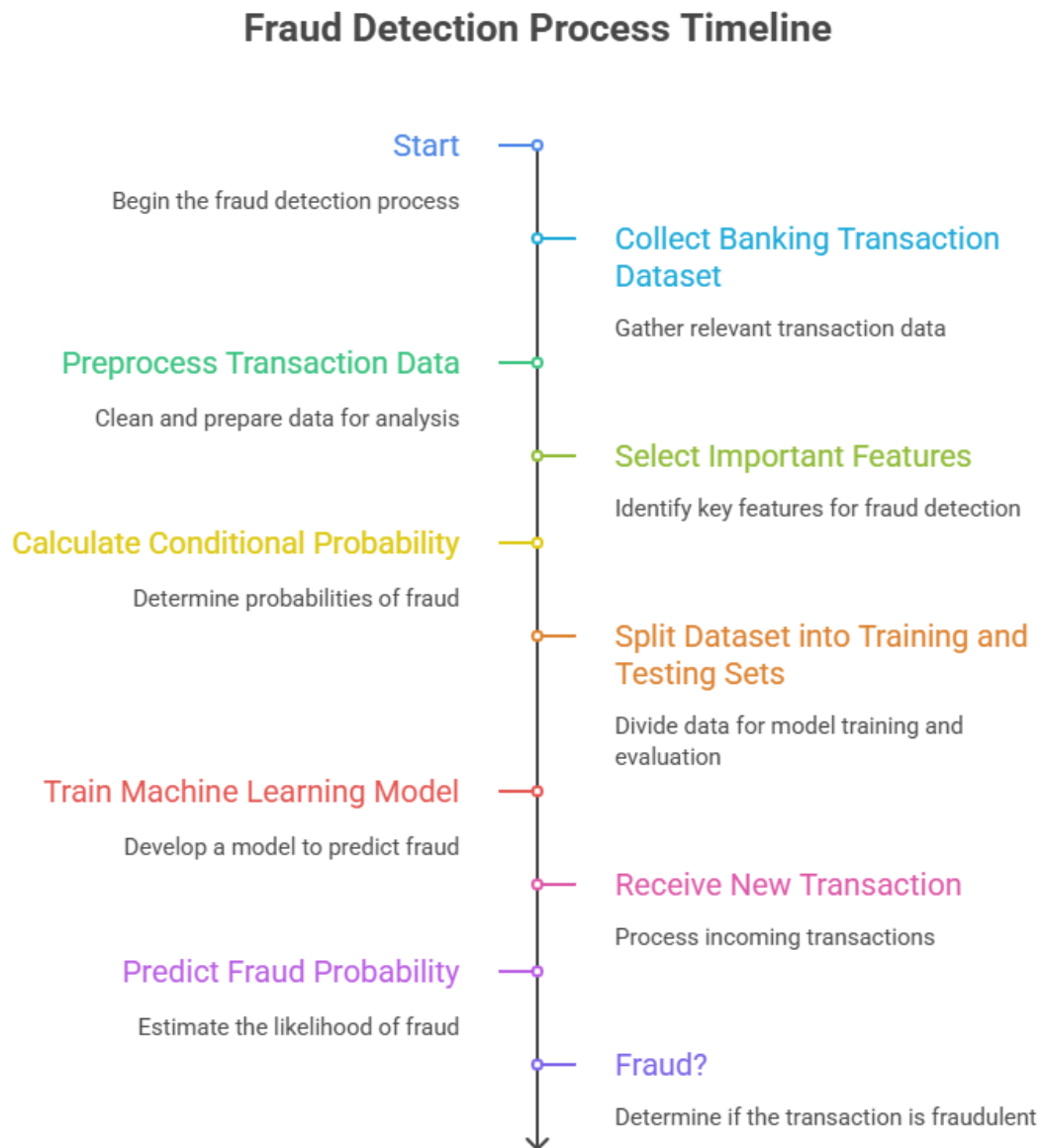
Sample Output:

Transaction Status: Fraud

Fraud Probability: 0.94

Model Accuracy: 97.1%

Flowchart:



Explanation of the Flowchart:

- **Start**: Begin the fraud detection process.
- **Collect Dataset**: Load historical banking transaction records.
- **Preprocess Data**: Clean and prepare the dataset.
- **Feature Selection**: Choose important fraud-related attributes.
- **Conditional Probability**: Estimate fraud likelihood based on transaction conditions.
- **Train Model**: Build the fraud detection model.
- **Receive Transaction**: Input a new banking transaction.
- **Prediction**: Calculate fraud probability.

- Decision: If fraud probability is high, block or flag the transaction; otherwise, approve it.
- Display Result: Show the classification outcome.
- End: Complete the process.

Advantages:

- Detects fraudulent transactions quickly and accurately.
- Reduces financial losses for banks and customers.
- Supports real-time transaction monitoring.
- Improves banking security and customer trust.
- Learns from historical transaction patterns.
- Can handle millions of transactions efficiently.
- Minimizes manual verification efforts.
- Helps financial institutions comply with security regulations.

Limitations:

- Requires a large and well-labeled dataset for effective learning.
- Fraud patterns evolve over time, requiring periodic model updates.
- Incorrectly flagged genuine transactions (false positives) may inconvenience customers.
- Highly imbalanced datasets can affect prediction performance.
- Complex fraud techniques may require more advanced machine learning models.

Real-World Applications:

- Credit Card Fraud Detection Systems
- Internet Banking Security
- Mobile Banking Applications
- ATM Transaction Monitoring
- Digital Wallet Fraud Prevention
- Online Payment Gateway Security
- Financial Risk Management
- Banking Compliance and Anti-Money Laundering (AML) Systems

Conclusion:

Conditional Probability plays a vital role in modern fraud detection systems by estimating the likelihood of fraudulent transactions based on observed conditions. By analysing transaction attributes such as amount, location, time, and previous fraud history, machine learning models can effectively distinguish between genuine and suspicious activities. The proposed fraud detection system follows a structured process of data collection, preprocessing, feature selection, conditional probability calculation, model training, prediction, and evaluation. Such systems significantly improve banking security, reduce financial fraud, and enhance customer confidence by enabling timely detection and prevention of fraudulent transactions.

Q9) Simulate a Real-World Stock Market Prediction Problem Using Random Variables and Suitable Probability Distributions

A)

Introduction:

The stock market is one of the most dynamic financial systems where the prices of stocks fluctuate continuously due to various economic, political, and market-related factors. Investors and financial analysts use historical stock prices, trading volumes, and technical indicators to predict future market movements. Since stock prices are uncertain and influenced by random events, Random Variables and Probability Distributions are widely used to model stock price behaviour.

Machine Learning further enhances stock market prediction by learning patterns from historical market data and estimating the probability of future price increases or decreases. By combining probability theory with predictive algorithms, investors can make informed investment decisions while managing financial risks.

The objective of this system is to simulate a real-world stock market prediction model using random variables and probability distributions to estimate future stock price movements.

Dataset Used:

Dataset Name: Stock Market Prediction Dataset

Source: Kaggle – Stock Market Dataset / Yahoo Finance Historical Stock Data

Dataset Link: <https://www.kaggle.com/datasets/jacksoncrow/stock-market-dataset>

Dataset Description:

The dataset contains historical stock market information collected from publicly traded companies.

Target Variable:

Stock Price Movement:

Possible Classes:

- Increase
- Decrease

Machine Learning Concepts:

Random Variable:

Definition:

A Random Variable is a variable whose value depends on the outcome of a random event. In stock market prediction, future stock prices cannot be known with certainty because they depend on many unpredictable factors such as market demand, company performance, economic conditions, inflation, and investor sentiment.

Examples of random variables in stock market analysis include:

- Daily closing price
- Daily return
- Trading volume
- Market index
- Percentage price change

Types of Random Variables:

1. Discrete Random Variable

Represents countable outcomes.

Examples:

- Stock price increases
- Stock price decreases

2. Continuous Random Variable

Represents measurable numerical values.

Examples:

- Stock price
- Daily return
- Trading volume
- Percentage gain/loss

Probability Distribution:

Definition:

A Probability Distribution describes the likelihood of different values taken by a random variable. It helps estimate the probability of stock prices increasing or decreasing.

Historical stock prices are analyzed to estimate future market behavior.

Probability Formula:

The probability of an event is calculated as:

$$P(E) = \frac{\text{Number of Favorable Outcomes}}{\text{Total Number of Outcomes}}$$

Where:

- $P(E)$ = Probability of event E
- Favorable Outcomes = Number of successful events
- Total Outcomes = Total observations

Example:

Suppose historical data contains:

- Total Trading Days = 250
- Price Increased = 160 Days

Then,

$$P(\text{Increase}) = \frac{160}{250} = 0.64$$

Therefore,

Probability of Price Increase = 64%

Normal Distribution:

Many financial analysts assume that stock returns approximately follow a Normal Distribution, where most daily returns lie close to the average while extreme gains and losses occur less frequently.

The Normal Distribution is represented by:

$$X \sim N(\mu, \sigma^2)$$

Where:

- μ = Mean Return
- σ^2 = Variance

This distribution helps estimate future price behavior and investment risk.

Working Principle:

The stock market prediction system follows these stages:

Step 1: Collect historical stock market data.

Step 2: Preprocess the dataset by removing missing values.

Step 3: Select important financial features.

Step 4: Represent stock prices as random variables.

Step 5: Calculate probability distributions using historical prices.

Step 6: Train the machine learning model.

Step 7: Predict future stock price movement.

Step 8: Evaluate model performance.

Methodology:

Step 1 – Data Collection

Collect historical stock market information from financial databases.

The dataset includes:

- Opening Price
- Closing Price
- High Price
- Low Price
- Trading Volume
- Market Index

Step 2 – Data Preprocessing

Before training:

- Remove duplicate records.
- Handle missing values.
- Normalize numerical features.
- Remove inconsistent entries.

This improves data quality and prediction performance.

Step 3 – Feature Selection

Select important financial indicators such as:

- Open Price
- Close Price
- High Price
- Low Price
- Trading Volume

These features strongly influence stock price movements.

Step 4 – Probability Modeling

Treat stock prices and daily returns as random variables.

Estimate:

- Probability of Price Increase
- Probability of Price Decrease
- Expected Daily Return

These probabilities assist in predicting future trends.

Step 5 – Model Training

Train a machine learning model using algorithms such as:

- Random Forest
- Decision Tree
- Support Vector Machine (SVM)
- Linear Regression (for price prediction)
- Logistic Regression (for movement classification)

The model learns relationships between historical prices and future movements.

Step 6 – Prediction

Example Input:

Open Price = ₹2,450

High Price = ₹2,520

Low Price = ₹2,430

Volume = 1,250,000

Predicted Output:

Stock Price Likely to Increase

Step 7 – Evaluation

Evaluate the prediction using:

- Accuracy
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score

These metrics indicate how well the model predicts future stock prices.

Algorithm (Pseudocode):

Stock Market Prediction Using Random Variables and Probability Distribution

1. Start the program.
2. Load the Stock Market dataset.
3. Read historical stock prices and trading information.
4. Remove missing and duplicate records.
5. Select important financial features.
6. Represent stock prices as random variables.
7. Calculate probability distributions from historical data.
8. Split the dataset into training and testing sets.
9. Train the machine learning model.
10. Predict future stock price movement.
11. Evaluate the prediction using performance metrics.
12. Display prediction results.
13. Stop the program.

Python Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```

# Load dataset
data = pd.read_csv("stock_market.csv")

# Input features
X = data[["Open","High","Low","Volume"]]

# Target variable
y = data["Movement"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Train model
model = RandomForestClassifier()
model.fit(X_train, y_train)

# Prediction
prediction = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, prediction)
print("Accuracy:", accuracy)

```

Sample Input:

Open Price	High Price	Low Price	Volume
₹2,450	₹2,520	₹2,430	1,250,000

Sample Output:

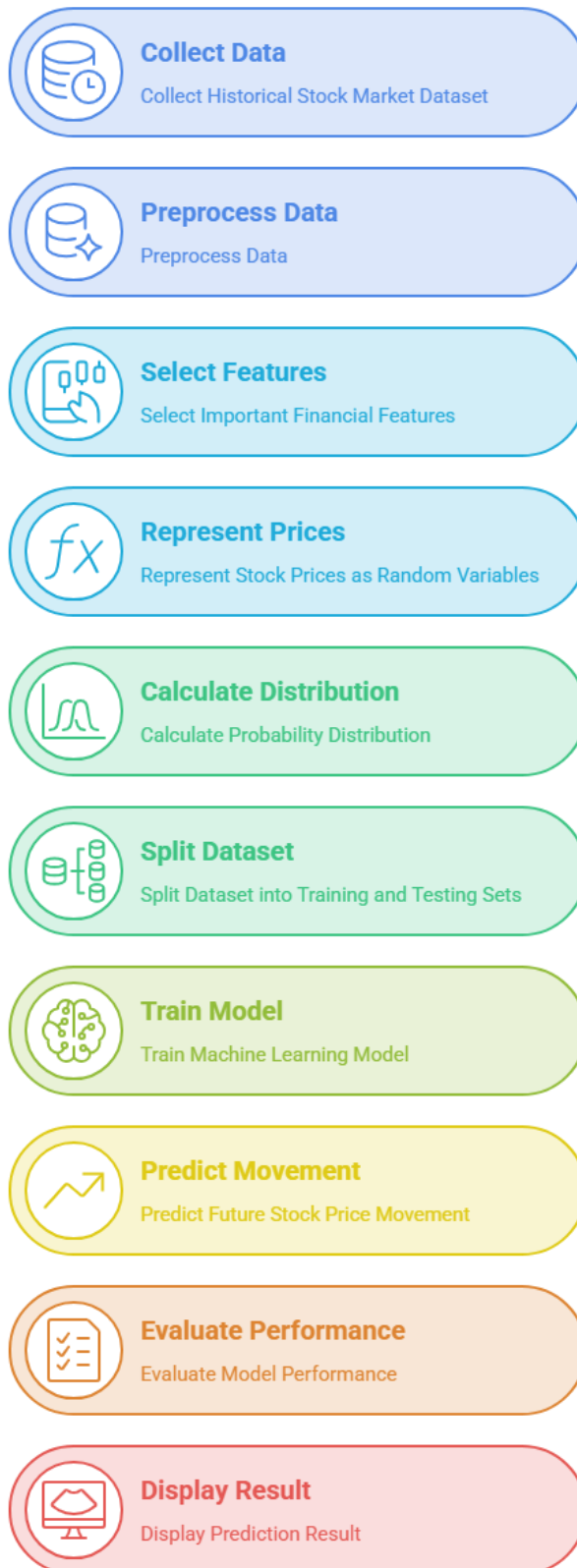
Predicted Movement: Increase

Probability of Increase: 0.81

Model Accuracy: 96.3%

Flowchart:

Stock Market Prediction Process



Explanation of the Flowchart:

- Start: Begin the stock market prediction process.
- Collect Dataset: Load historical stock market data.
- Preprocess Data: Clean and prepare the dataset.
- Feature Selection: Choose significant financial indicators.
- Random Variables: Represent stock prices and returns mathematically.
- Probability Distribution: Estimate the likelihood of future price movements.
- Train Model: Build the machine learning prediction model.
- Prediction: Forecast whether the stock price will increase or decrease.
- Evaluation: Measure prediction performance using accuracy and error metrics.
- Display Result: Show the predicted stock movement and probability.
- End: Complete the prediction process.

Advantages:

- Assists investors in making informed investment decisions.
- Reduces investment risk through probability-based forecasting.
- Analyses large volumes of historical financial data efficiently.
- Supports automated trading and portfolio management.
- Improves prediction accuracy using machine learning algorithms.
- Helps identify market trends and investment opportunities.
- Can continuously improve with updated market data.
- Provides valuable insights for financial analysts.

Limitations:

- Stock markets are highly volatile and influenced by unpredictable events.
- Economic and political changes can reduce prediction accuracy.
- Historical performance does not guarantee future results.
- Requires large amounts of quality financial data.
- Frequent model retraining is necessary to adapt to changing market conditions.

Real-World Applications:

- Stock Exchange Trading Platforms

- Investment Advisory Systems
- Portfolio Management Applications
- Algorithmic Trading Systems
- Financial Risk Analysis
- Banking and Financial Institutions
- Hedge Fund Market Analysis
- Robo-Advisory Investment Platforms

Conclusion:

Stock market prediction using random variables and probability distributions provides a scientific approach to understanding uncertain market behaviour. By representing stock prices as random variables and estimating the probability of future price movements from historical data, machine learning models can assist investors in making informed financial decisions. The proposed system follows a structured workflow consisting of data collection, preprocessing, feature selection, probability modelling, model training, prediction, and evaluation. Although market volatility and unforeseen events can affect prediction accuracy, probability-based machine learning models remain valuable tools for investment analysis, risk management, and intelligent financial decision-making.